

Loopline P2P Application - Complete Technical Documentation

Version: 1.0

Last Updated: May 25, 2026

Author: Development Team

Table of Contents

1. [System Overview](#)
 2. [Architecture](#)
 3. [Backend Documentation](#)
 4. [Frontend Documentation](#)
 5. [API Endpoints](#)
 6. [Data Flow](#)
 7. [Configuration](#)
 8. [Deployment Guide](#)
 9. [Troubleshooting](#)
-

System Overview

Loopline is a peer-to-peer (P2P) file transfer and synchronization application with a distributed architecture. It consists of:

- **Backend:** C++ server handling file transfers, HTTP API, and peer coordination
- **Frontend:** User interface for file management and peer interaction

The system enables secure file sharing between multiple peers with real-time synchronization of shared folders.

Key Features

- HTTP REST API for file operations
- Direct P2P file transfer between nodes

- Real-time shared folder synchronization
 - Multi-peer network support
 - Cross-platform compatibility (Windows, Linux, macOS)
 - Configurable network parameters
 - Atomic operations for state management
-

Architecture

High-Level System Diagram

CLIENT MACHINES

Frontend UI
(React/Vue/etc)

Frontend UI
(React/Vue/etc) ...

HTTP Client Library
(Axios/Fetch/etc)

HTTP Requests (Port 8787)

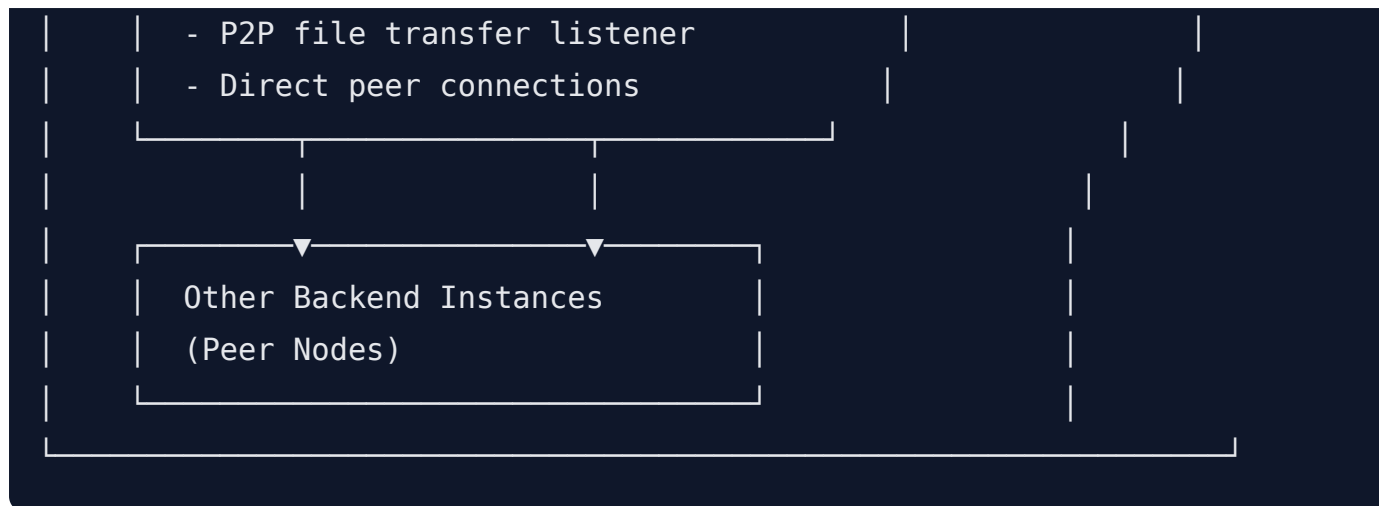
Backend Server (C++)

HTTP Server (Port 8787)
- HttpController
- Route HTTP requests

File Transfer Shared AppState
Vault Service Sync Manager
Service Service

recv sent share config
dirs dirs dir files

Transfer Server (Port 8788)



Architecture

Component Diagram

AppState (Central State)

- └─ http_port
- └─ transfer_port
- └─ bind_host
- └─ advertised_host
- └─ allow_remote_peers
- └─ directories (receive, sent, shared)
- └─ sync_peers[]
- └─ running (atomic)
- └─ listener_active (atomic)

HttpServer (Main Loop)

- └─ Create Listener Socket
- └─ Setup Dependencies (Lambda Functions)
- └─ Accept Loop
 - └─ Accept Connection
 - └─ Create Handler Thread
 - └─ Detach Thread

HttpController

- └─ status_json()
- └─ file_list_json()
- └─ send_file_inline()
- └─ send_file_to_peer()
- └─ sync_shared_folder_json()
- └─ save_shared_upload()
- └─ add_sync_peer()
- └─ remove_sync_peer()
- └─ is_allowed_peer()

FileVaultService

- └─ file_list_json()
- └─ send_file_inline()
- └─ save_shared_upload()

TransferService

- └─ transfer_listener() (Background Thread)

```
|— send_file_to_peer()
|— receive_file()
```

SharedSyncService

```
|— watch_shared_folder() (Background Thread)
|— sync_shared_folder_json()
```

Backend Documentation

Backend Class Definition

```
// filepath: backend/src/app/backend_app.hpp
#pragma once

class BackendApp {
public:
    int run(int argc, char* argv[]);
};
```

Purpose: Entry point for the backend application

Method:

- `run(argc, argv)` - Main application entry, initializes and starts all services

File Structure

```

backend/
├─ src/
│   ├─ app/
│   │   ├─ backend_app.hpp      (Main class definition)
│   │   └─ backend_app.cpp      (Main implementation)
│   ├─ controllers/
│   │   └─ http_controller.hpp  (Request routing)
│   ├─ models/
│   │   └─ app_state.hpp        (State management)
│   ├─ services/
│   │   ├─ file-vault/
│   │   │   └─ file_vault_service.hpp
│   │   ├─ net-io/
│   │   │   └─ net_io.hpp        (Socket operations)
│   │   ├─ shared-sync/
│   │   │   └─ shared_sync_service.hpp
│   │   └─ transfer/
│   │       └─ transfer_service.hpp (P2P transfers)
│   └─ views/
│       └─ http_view.hpp         (HTTP responses)
├─ received/                    (Downloaded files)
└─ sent/                       (Uploaded files)

```

Configuration

Environment Variables

Variable	Type	Default	Description
P2P_HTTP_PORT	int	8787	HTTP API server port
P2P_TRANSFER_PORT	int	8788	P2P transfer server port
P2P_BIND_HOST	string	0.0.0.0	Network interface to bind
P2P_ADVERTISED_HOST	string	{bind_host}	Host advertised to peers
P2P_ALLOW_REMOTE	bool	true	Allow remote peer connections

P2P_RECEIVE_DIR	path	./backend/received	Downloaded files directory
P2P_SENT_DIR	path	./backend/sent	Uploaded files directory
P2P_SHARED_DIR	path	./shared	Shared folder for sync
P2P_SYNC_PEERS	string	(empty)	Initial peers (comma/semicolon-separated)

Command-Line Arguments

Arguments override environment variables with higher priority:

```
./backend_app \  
  --http 8787 \  
  --transfer 8788 \  
  --bind 0.0.0.0 \  
  --advertise 127.0.0.1
```

Argument	Type	Example	Description
--http	int	8787	HTTP port override
--transfer	int	8788	Transfer port override
--bind	string	0.0.0.0	Bind interface override
--advertise	string	192.168.1.100	Public host override

Backend Components

1. AppState Model

Centralized application state container.

Responsibilities:

- Store configuration values
- Manage peer list
- Maintain server status flags
- Generate status JSON

Key Members:

```
int http_port;
int transfer_port;
std::string bind_host;
std::string advertised_host;
bool allow_remote_peers;
std::filesystem::path receive_dir;
std::filesystem::path sent_dir;
std::filesystem::path shared_dir;
std::atomic<bool> running;
std::atomic<bool> listener_active;
std::vector<transfer::PeerEndpoint> sync_peers;
```

Methods:

- `status_json()` - Returns JSON representation of state
 - `add_sync_peer(PeerEndpoint)` - Add peer to network
 - `remove_sync_peer(PeerEndpoint)` - Remove peer from network
-

2. FileVaultService

Manages file storage and retrieval operations.

Responsibilities:

- List files by category (received, sent, shared)
- Send files to HTTP clients
- Handle file uploads
- Generate file list JSON

Methods:

- `file_list_json(kind)` - Get files of type (received/sent/shared)
- `send_file_inline(socket, kind, name)` - Stream file to HTTP client
- `save_shared_upload(socket, name, body)` - Store uploaded file
- `send_file_to_peer(socket, host, port, name, body)` - Forward to peer

Directory Structure:

```
backend/  
├─ received/      (Downloaded from peers)  
├─ sent/          (Uploaded by this node)  
└─ shared/        (Synchronized across network)
```

3. TransferService

Handles peer-to-peer file transfers.

Responsibilities:

- Listen for incoming P2P connections (port 8788)
- Receive files from remote peers
- Send files to remote peers
- Manage peer connections

Methods:

- `transfer_listener()` - Accept incoming peer connections
- `send_file_to_peer(...)` - Send file to specific peer
- `receive_file(socket, name)` - Handle file from peer

Peer Connection Flow:

1. Client connects to transfer port
 2. Service receives peer info
 3. File data is transmitted
 4. File stored in received directory
 5. Connection closed
-

4. SharedSyncService

Monitors and synchronizes shared folder.

Responsibilities:

- Watch shared folder for changes
- Detect new/modified/deleted files
- Generate sync state JSON
- Notify other peers of changes

Methods:

- `watch_shared_folder()` - Main monitoring loop
- `sync_shared_folder_json()` - Get current state

Watched Events:

- File creation
 - File modification
 - File deletion
-

5. HttpController

Routes HTTP requests to appropriate handlers.

Dependencies (Passed via Lambda Closures):

```
struct Dependencies {
    std::function<std::string()> status_json;
    std::function<std::string(std::string)> file_list_json;
    std::function<void(socket_t, std::string, std::string)>
send_file_inline;
    std::function<void(socket_t, std::string, int, std::string,
std::vector<char>)> send_file_to_peer;
    std::function<std::string()> sync_shared_folder_json;
    std::function<void(socket_t, std::string, std::vector<char>)>
save_shared_upload;
    std::function<int()> transfer_port;
    std::function<bool()> listener_active;
    std::function<void(transfer::PeerEndpoint)> add_sync_peer;
    std::function<void(transfer::PeerEndpoint)> remove_sync_peer;
    std::function<bool(std::string)> is_allowed_peer;
};
```

Request Flow:

1. Accept HTTP connection
2. Parse HTTP request
3. Route to handler
4. Call appropriate dependency lambda
5. Format response with `HttpView`
6. Send response
7. Close connection

Utility Functions

`lower_copy(std::string value) → std::string`

Converts ASCII uppercase letters to lowercase.

Usage: Environment variable case-insensitive parsing

Example:

```
lower_copy("TRUE") → "true"  
lower_copy("Yes") → "yes"
```

`parse_int(const std::string& value) → std::optional<int>`

Parses string to integer using `std::from_chars`.

Validation:

- Entire string must be valid integer
- No partial matches accepted

Usage: Port number parsing, configuration values

Example:

```
parse_int("8787") → 8787  
parse_int("invalid") → std::nullopt
```

```
env_value(const char* name, const std::string& fallback) → std::string
```

Retrieves environment variable with fallback.

Logic:

1. Call `std::getenv(name)`
2. If NULL or empty, return fallback
3. Otherwise return environment value

Usage: Configuration loading

Example:

```
env_value("P2P_HTTP_PORT", "8787") → "8787" (or env value if set)
```

```
env_flag(const char* name, bool fallback) → bool
```

Parses environment variable as boolean flag.

Recognized True Values: "1", "true", "yes", "on" (case-insensitive)

Usage: Feature flags, boolean configuration

Example:

```
env_flag("P2P_ALLOW_REMOTE", true) → true if env="1" or "true"
```

```
arg_value(int argc, char* argv[], const std::string& name, int fallback) → int
```

Retrieves integer command-line argument.

Lookup Strategy:

1. Find `argv[index] == name`
2. Parse `argv[index+1]` as integer
3. Return parsed value or fallback

Usage: Port overrides, numeric configuration

Example:

```
Command: ./app --http 9999  
arg_value(..., "--http", 8787) → 9999
```

arg_string(int argc, char* argv[], const std::string& name, const std::string& fallback) → std::string

Retrieves string command-line argument.

Lookup Strategy:

1. Find argv[index] == name
2. Return argv[index+1]
3. Return fallback if not found

Usage: Host configuration

Example:

```
Command: ./app --bind 192.168.1.1  
arg_string(..., "--bind", "0.0.0.0") → "192.168.1.1"
```

add_sync_peers_from_list(AppState& state, const std::string& raw_peers) → void

Parses peer list and registers them.

Peer Format: host:port separated by comma or semicolon

Validation:

- Parse endpoint using transfer::parse_peer_endpoint()
- Validate with transfer::is_allowed_peer()
- Only add if both pass

Delimiter Priority: Semicolon > Comma

Example:

```
Input: "peer1.com:8788,peer2.com:8788;192.168.1.5:8788"
```

```
Result: Three peers registered
```

```
http_server(AppState&, FileVaultService&, TransferService&,
SharedSyncService&) → void
```

Main HTTP server implementation.

Execution Flow:

1. Socket Creation

- Create listener socket on `bind_host:http_port`
- Log error if creation fails, return

2. Dependency Setup

- Create lambdas for each controller dependency
- Capture services by reference

3. Controller Initialization

- Create shared `HttpController` instance
- Pass dependencies to controller

4. Log Startup

- Print HTTP API URL to console

5. Accept Loop

- While `state.running` is true:
 - Accept incoming connection
 - Create client handler thread
 - Detach thread (fire-and-forget)
 - Continue accepting

6. Cleanup

- Close listener socket on exit

Thread Model:

- One thread per accepted connection
 - Detached threads (independent lifecycle)
 - No explicit thread joining
-

Main Application Flow

```
BackendApp::run(int argc, char* argv[]) → int
```

Step-by-Step Execution:

Windows Initialization (Windows Only)

```
WSADATA data{};
WSAStartup(MAKEWORD(2, 2), &data);
```

- Initialize Windows Sockets API
- Required on Windows platforms
- Cleaned up at exit with `WSACleanup()`

Configuration Loading

Priority Order:

1. Command-line arguments (highest priority)
2. Environment variables
3. Hardcoded defaults (lowest priority)

Configuration Chain:

```
state.http_port = arg_value(argc, argv, "--http",
    parse_int(env_value("P2P_HTTP_PORT", "8787")).value_or(8787));

// This evaluates as:
// 1. Check --http argument
// 2. If not found, check P2P_HTTP_PORT env
// 3. If not found, use "8787"
```

Loaded Configuration:

```
http_port      : Command-line or ENV or 8787
transfer_port  : Command-line or ENV or 8788
bind_host      : Command-line or ENV or "0.0.0.0"
advertised_host : Command-line or ENV or bind_host
allow_remote   : ENV or true
receive_dir    : ENV or ./backend/received
sent_dir       : ENV or ./backend/sent
shared_dir     : ENV or ./shared
sync_peers     : Parsed from ENV P2P_SYNC_PEERS
```

Address Adjustment

```
if (state.advertised_host == "0.0.0.0") {
    state.advertised_host = "127.0.0.1";
}
```

- Wildcard bind address cannot be advertised
- Automatically changes to localhost

Directory Creation

```
std::filesystem::create_directories(state.receive_dir);
std::filesystem::create_directories(state.sent_dir);
std::filesystem::create_directories(state.shared_dir);
```

- Creates all required directories
- Creates parent directories if needed
- Silently succeeds if already exists

Service Initialization

```
FileVaultService file_vault_service(state);
SharedSyncService shared_sync_service(state);
TransferService transfer_service(state, shared_sync_service);
```

- Create service instances
- Pass state for configuration

- TransferService needs SharedSyncService

Background Threads

Transfer Listener Thread:

```
std::thread receiver(&TransferService::transfer_listener,  
&transfer_service);  
receiver.detach();
```

- Listens on transfer_port for P2P connections
- Runs independently
- Detached (not joined)

Shared Folder Watcher Thread:

```
std::thread shared_watcher(&SharedSyncService::watch_shared_folder,  
&shared_sync_service);  
shared_watcher.detach();
```

- Monitors shared_dir for changes
- Runs independently
- Detached (not joined)

Console Output

```
Loopline P2P receiver: 127.0.0.1:8788  
Bind host: 0.0.0.0 | remote peers: enabled  
Received files: /path/to/backend/received  
Sent files: /path/to/backend/sent  
Shared folder: /path/to/shared  
HTTP API: http://127.0.0.1:8787
```

Main Loop

```
http_server(state, file_vault_service, transfer_service,  
shared_sync_service);
```

- Blocking call, runs until shutdown
- Accepts HTTP connections
- Spawns handler threads

Cleanup

```
#ifdef _WIN32
    WSACleanup();
#endif
return 0;
```

- Windows Sockets cleanup
 - Return success code
-

Frontend Documentation

Purpose

The frontend provides a user-friendly interface for:

- Viewing available files
- Uploading files to peers
- Downloading files from peers
- Managing peer connections
- Monitoring shared folder synchronization

Technology Stack Expectations

The frontend should connect to the backend HTTP API (port 8787) and implement:

- **Framework:** React, Vue, Angular, or similar SPA framework
- **HTTP Client:** Axios, Fetch API, or similar
- **UI Components:** Material-UI, Bootstrap, or custom CSS
- **State Management:** Redux, Vuex, Context API, or similar
- **Build Tool:** Webpack, Vite, or similar

Frontend Architecture

```
frontend/
├─ public/
│   └─ index.html
├─ src/
│   ├─ components/
│   │   ├─ FileList.tsx          (Display files)
│   │   ├─ PeerManager.tsx      (Add/remove peers)
│   │   ├─ SharedFolderStatus.tsx (Sync status)
│   │   └─ StatusPanel.tsx      (System status)
│   ├─ services/
│   │   └─ api.ts                (HTTP client)
│   ├─ pages/
│   │   ├─ Dashboard.tsx        (Main page)
│   │   ├─ Downloads.tsx        (Received files)
│   │   ├─ Uploads.tsx          (Sent files)
│   │   └─ Shared.tsx           (Shared folder)
│   ├─ hooks/
│   │   ├─ useFileList.ts       (File fetching)
│   │   ├─ usePeers.ts          (Peer management)
│   │   └─ useStatus.ts         (Status polling)
│   ├─ types/
│   │   └─ api.ts               (TypeScript interfaces)
│   └─ App.tsx                  (Root component)
└─ package.json
```

Frontend Components

1. API Service Module

Handles all HTTP communication with backend.

Base URL: `http://localhost:8787` (configurable)

Methods:

```
// Get system status
getStatus(): Promise<StatusData>

// File operations
getReceivedFiles(): Promise<FileList>
getSentFiles(): Promise<FileList>
getSharedFiles(): Promise<FileList>

// File transfer
downloadFile(kind: string, filename: string): Promise<Blob>
uploadFile(filename: string, file: File): Promise<Response>
sendFileToPeer(host: string, port: number, filename: string, file:
File): Promise<Response>

// Peer management
getPeers(): Promise<PeerList>
addPeer(host: string, port: number): Promise<Response>
removePeer(host: string, port: number): Promise<Response>

// Shared folder
getSharedFolderStatus(): Promise<SyncStatus>
```

2. Status Panel Component

Displays real-time system information.

Displays:

- HTTP server status and address
- Transfer server status and address
- Bind host configuration
- Remote peers allowed status
- Connected peers count
- Uptime/Last sync time

Refresh Rate: 5-10 seconds polling

Data Source: `GET /api/status`

3. File List Component

Shows available files in categories.

Tabs/Views:

- **Received:** Files downloaded from peers
- **Sent:** Files uploaded by this node
- **Shared:** Files in synchronized folder

For Each File:

- Filename
- File size
- Modified date
- Action buttons (download/delete/share)

Data Source:

- `GET /api/files/received`
- `GET /api/files/sent`
- `GET /api/files/shared`

Upload Functionality:

- Drag-and-drop file upload
- File browser selection
- Progress indicator
- Success/error notification

Download Functionality:

- Click to download
 - Shows progress
 - Browser default download handler
-

4. Peer Manager Component

Manages peer connections.

Display:

- List of connected peers
- Peer host/port
- Last sync time
- Connection status

Actions:

- Add new peer (input host:port)
- Remove peer (confirmation)
- Manual sync trigger

Data Source:

- GET /api/peers
- POST /api/peers (add)
- DELETE /api/peers/:id (remove)

Input Validation:

- Host format validation (IP or domain)
 - Port range validation (1-65535)
 - Duplicate check
-

5. Shared Folder Sync Component

Displays synchronized folder status.

Displays:

- Last sync time
- Sync status (synced/syncing/error)
- File count in shared folder
- Recent changes log
- Sync health indicator

Data Source: GET /api/shared/status

Refresh Rate: 5 seconds polling

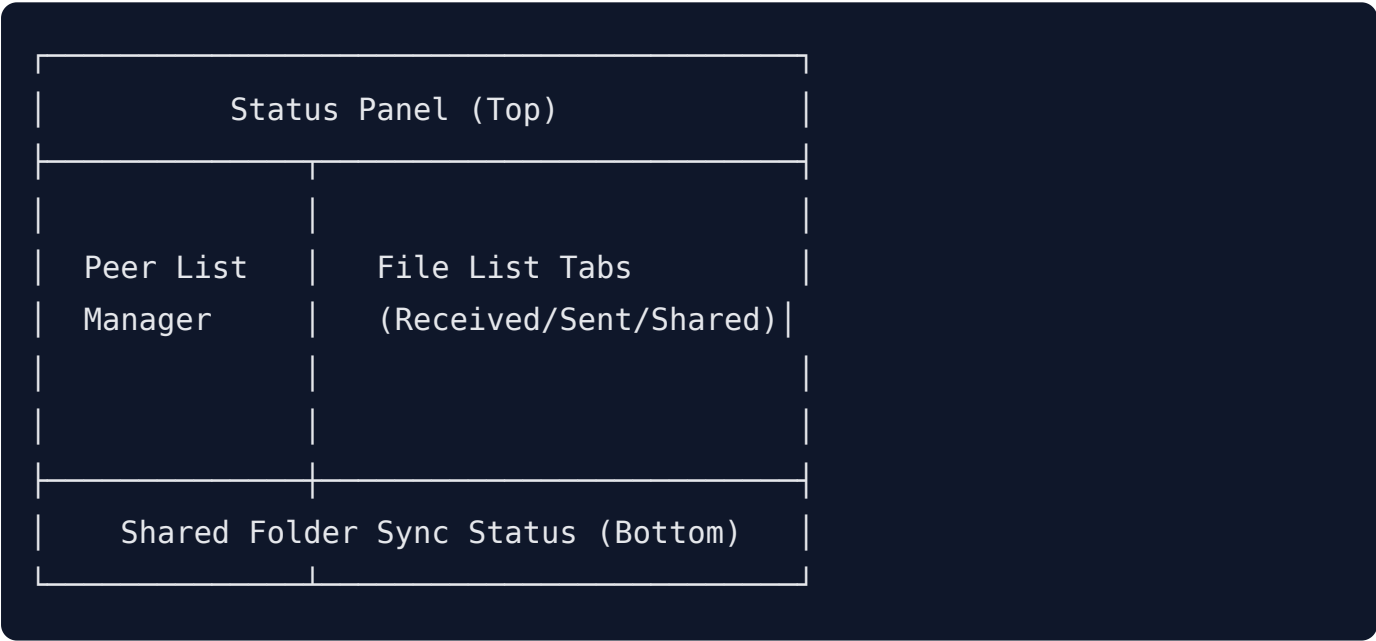
File List:

- All files in shared folder
- Size
- Last modified
- Sync state

6. Dashboard Page

Main landing page combining all components.

Layout:



Frontend State Management

Key State Values

```
interface AppState {  
  // Status  
  serverStatus: StatusData  
  isConnected: boolean  
  lastUpdate: Date  
  
  // Files  
  receivedFiles: File[]  
  sentFiles: File[]  
  sharedFiles: File[]  
  loadingFiles: boolean  
  
  // Peers  
  peers: PeerEndpoint[]  
  selectedPeer: PeerEndpoint | null  
  loadingPeers: boolean  
  
  // Shared Sync  
  syncStatus: SyncStatus  
  lastSyncTime: Date  
  syncEnabled: boolean  
  
  // UI State  
  currentTab: 'received' | 'sent' | 'shared'  
  uploadProgress: number  
  notifications: Notification[]  
}
```

State Update Triggers

- **Auto-refresh:** Status/file list/peer list every 5-10 seconds
- **User action:** File upload/download, peer add/remove
- **WebSocket (Optional):** Real-time updates for better UX
- **Manual refresh:** User-triggered force refresh

Frontend User Workflows

Workflow 1: Download File from Peer

1. User selects "Received Files" tab
2. Frontend fetches received file list (GET /api/files/received)
3. User clicks download button on file
4. Frontend initiates download (GET /api/files/received/{filename})
5. Browser saves file
6. User sees success notification

Workflow 2: Upload File

1. User drags file to upload zone
2. Frontend validates file
3. User confirms upload
4. Frontend sends file (POST /api/files/upload)
5. Progress bar shows transfer status
6. Success notification shown
7. File appears in "Sent Files" tab after refresh

Workflow 3: Add New Peer

1. User clicks "Add Peer" button in Peer Manager
2. Modal/form appears asking for host and port
3. User enters peer address (e.g., 192.168.1.5:8788)
4. Frontend validates input
5. User confirms
6. Frontend sends request (POST /api/peers)
7. Peer appears in peer list
8. Sync begins automatically

Workflow 4: Monitor Shared Folder Sync

1. User navigates to Shared folder tab
2. Frontend shows shared folder sync status
3. Frontend displays all files in shared folder
4. Auto-refresh shows updates every 5 seconds
5. Green indicator = synced, orange = syncing, red = error
6. User can see last sync time and file count

Workflow 5: Send File to Specific Peer

1. User selects file from Sent Files
2. User clicks "Send to Peer"
3. Peer selection dropdown appears
4. User selects destination peer
5. User confirms
6. Frontend sends request (POST /api/files/send-to-peer)
7. Progress shown
8. Success notification

Frontend Configuration

Environment Variables

```
REACT_APP_BACKEND_URL=http://localhost:8787
REACT_APP_POLL_INTERVAL=5000
REACT_APP_SOCKET_URL=ws://localhost:8789
REACT_APP_LOG_LEVEL=info
```

Local Storage

```
// User preferences
localStorage.setItem('theme', 'dark' | 'light')
localStorage.setItem('autoRefresh', true | false)
localStorage.setItem('pollInterval', 5000)

// Backend connection
localStorage.setItem('backendUrl', 'http://localhost:8787')
localStorage.setItem('lastConnected', timestamp)
```

API Endpoints

Status Endpoints

GET /api/status

```
Response:
{
  "http_port": 8787,
  "transfer_port": 8788,
  "bind_host": "0.0.0.0",
  "advertised_host": "127.0.0.1",
  "allow_remote_peers": true,
  "running": true,
  "listener_active": true,
  "sync_peers": [
    {"host": "peer1.com", "port": 8788},
    {"host": "192.168.1.5", "port": 8788}
  ]
}
```

GET /api/shared/sync-status

Response:

```
{
  "sync_status": "synced",
  "last_sync_time": "2026-05-25T12:05:00Z",
  "file_count": 15,
  "total_size": 104857600,
  "synced_peers": 2,
  "error_message": null
}
```

File Endpoints

GET /api/files/received

Response:

```
{
  "files": [
    {"name": "document.pdf", "size": 2048576, "modified": "2026-05-25T10:30:00Z"},
    {"name": "image.jpg", "size": 1024000, "modified": "2026-05-25T09:15:00Z"}
  ]
}
```

GET /api/files/sent

Response:

```
{
  "files": [
    {"name": "presentation.pptx", "size": 5242880, "modified": "2026-05-24T14:00:00Z"}
  ]
}
```

GET /api/files/shared

Response:

```
{
  "files": [
    {"name": "shared_doc.docx", "size": 1048576, "modified": "2026-05-25T11:45:00Z"}
  ]
}
```

POST /api/files/upload

Request: multipart/form-data with file

Response: {"status": "success", "filename": "uploaded_file.txt"}

GET /api/files/{kind}/{filename}

Response: File binary data

Headers: Content-Type, Content-Length, Content-Disposition

POST /api/files/send-to-peer

Request:

```
{
  "host": "192.168.1.5",
  "port": 8788,
  "filename": "document.pdf"
}
```

Response:

```
{
  "status": "success",
  "transfer_id": "abc123"
}
```

Peer Endpoints

GET /api/peers

Response:

```
{
  "peers": [
    {
      "id": 1,
      "host": "peer1.com",
      "port": 8788,
      "last_sync": "2026-05-25T12:00:00Z"
    },
    {
      "id": 2,
      "host": "192.168.1.5",
      "port": 8788,
      "last_sync": "2026-05-25T11:55:00Z"
    }
  ]
}
```

POST /api/peers

Request:

```
{  
  "host": "new-peer.com",  
  "port": 8788  
}
```

Response:

```
{  
  "status": "success",  
  "peer_id": 3  
}
```

DELETE /api/peers/{id}

Response:

```
{  
  "status": "success",  
  "removed_peer_id": 3  
}
```

Data Flow

Complete Request-Response Cycle

USER INTERACTION (Frontend)



| Parse JSON |

| Render UI |

File Download Flow

Frontend (Request)

|
└ GET /api/files/received/document.pdf
|

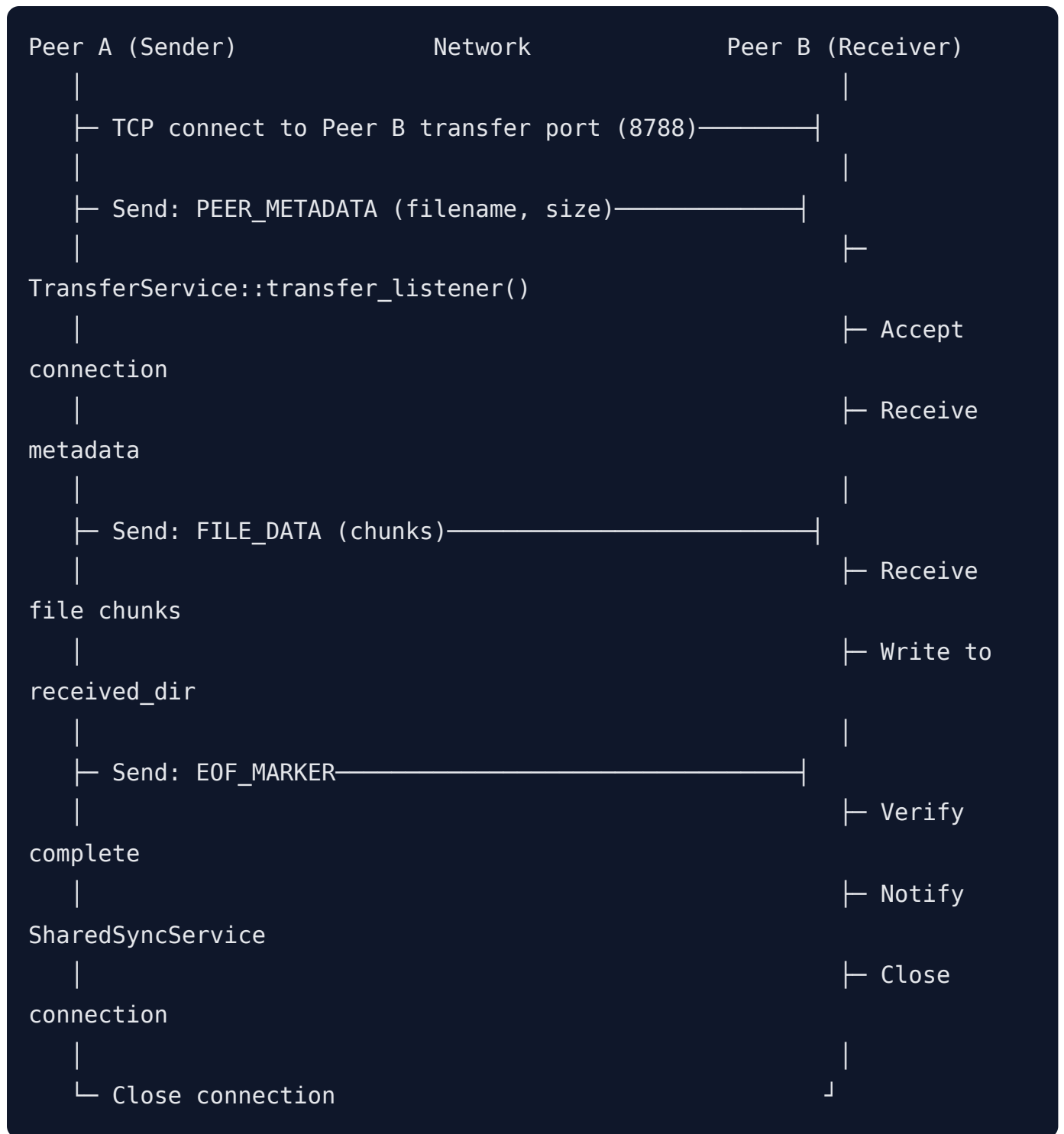
Backend HTTP Handler

|
└ Authenticate request
└ FileVaultService::send_file_inline()
| |
| └ Open file from received_dir
| └ Check file exists
| └ Send HTTP headers (Content-Type, Content-Length)
| └ Stream file contents
|
└ HttpView::format_response()
|

Frontend (Response)

|
└ Receive file binary data
└ Browser handles download
└ File saved to user's Downloads

P2P File Transfer Flow



Shared Folder Synchronization Flow

SharedSyncService

```
|
|├─ watch_shared_folder() loop (every N seconds)
|
|├─ Scan shared_dir for files
|
|├─ Detect changes:
|  |
|  |├─ New files: Add to peer queue
|  |├─ Modified files: Mark for resync
|  |└─ Deleted files: Remove from peer list
|
|├─ For each connected peer:
|  |
|  |├─ Connect to transfer port
|  |├─ Send changed files
|  |└─ Close connection
|
|└─ Update sync_status
```

Configuration

Complete Configuration Example

Environment Setup

```
# File: .env or environment variables

# HTTP Server
P2P_HTTP_PORT=8787
P2P_BIND_HOST=0.0.0.0
P2P_ADVERTISED_HOST=192.168.1.100

# Transfer Server
P2P_TRANSFER_PORT=8788
P2P_ALLOW_REMOTE=true

# Directories
P2P_RECEIVE_DIR=/var/loopleft/received
P2P_SENT_DIR=/var/loopleft/sent
P2P_SHARED_DIR=/var/loopleft/shared

# Initial Peers
P2P_SYNC_PEERS=peer1.company.com:8788;peer2.company.com:8788;192.168.1.5:8788
```

Command-Line Startup

```
# Windows
.\backend_app.exe ^
  --http 8787 ^
  --transfer 8788 ^
  --bind 0.0.0.0 ^
  --advertise 192.168.1.100

# Linux/macOS
./backend_app \
  --http 8787 \
  --transfer 8788 \
  --bind 0.0.0.0 \
  --advertise 192.168.1.100
```

Docker Configuration Example

```
FROM ubuntu:22.04

WORKDIR /app

COPY backend_app .
COPY shared /shared

ENV P2P_HTTP_PORT=8787
ENV P2P_TRANSFER_PORT=8788
ENV P2P_BIND_HOST=0.0.0.0
ENV P2P_ADVERTISED_HOST=loopleftline.example.com

EXPOSE 8787 8788

CMD [ "./backend_app" ]
```

Deployment Guide

Single-Node Deployment

System Requirements

- **OS:** Linux, macOS, or Windows
- **RAM:** 512MB minimum, 2GB recommended
- **Disk:** Depends on file size, minimum 100MB
- **Network:** TCP ports 8787, 8788 available

Installation Steps

1. Compile Backend

```
cd backend
mkdir build && cd build
cmake ..
make
```

2. Setup Directories

```
mkdir -p /var/loopleftine/{received,sent,shared}
chmod 755 /var/loopleftine/*
```

3. Deploy Frontend

```
cd frontend
npm install
npm run build
# Serve static files from dist/
```

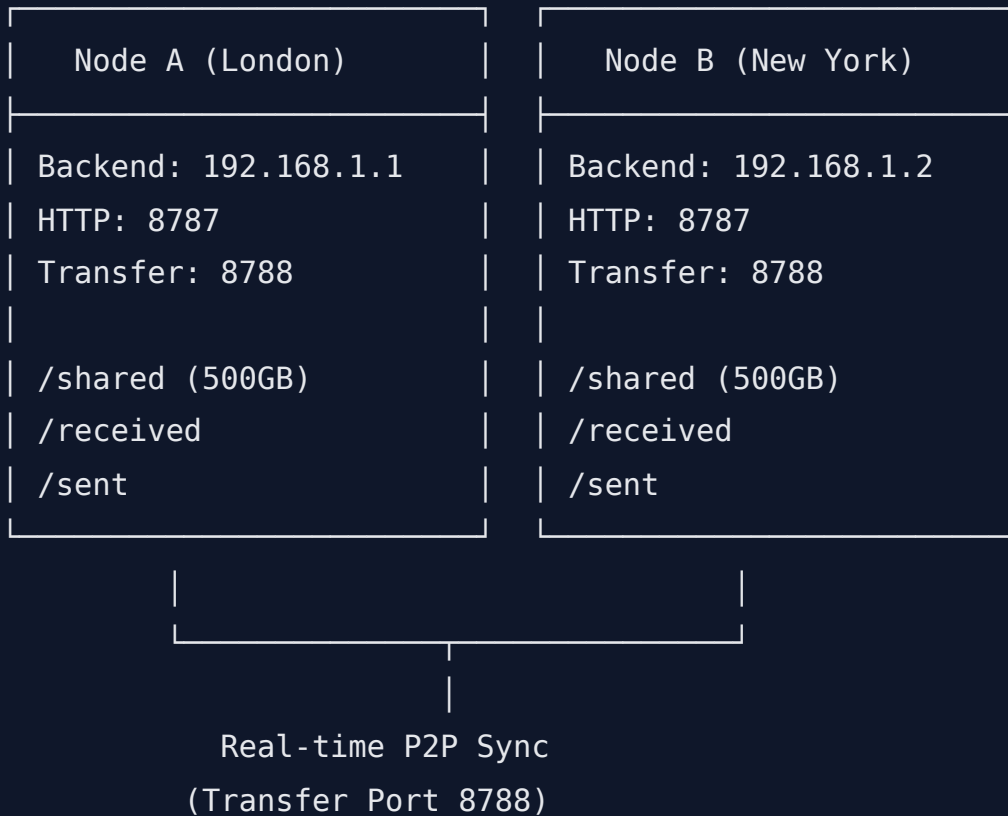
4. Start Backend

```
export P2P_HTTP_PORT=8787
export P2P_TRANSFER_PORT=8788
./backend_app
```

5. Access Frontend

- Open browser to `http://localhost:3000`
- Backend API available at `http://localhost:8787`

Multi-Node Cluster Deployment Architecture



Configuration for Node A

```
export P2P_HTTP_PORT=8787
export P2P_TRANSFER_PORT=8788
export P2P_BIND_HOST=0.0.0.0
export P2P_ADVERTISED_HOST=node-a.loopleft.internal
export P2P_SYNC_PEERS=node-b.loopleft.internal:8788
export P2P_SHARED_DIR=/mnt/nfs/shared
./backend_app
```

Configuration for Node B

```
export P2P_HTTP_PORT=8787
export P2P_TRANSFER_PORT=8788
export P2P_BIND_HOST=0.0.0.0
export P2P_ADVERTISED_HOST=node-b.loopleft.internal
export P2P_SYNC_PEERS=node-a.loopleft.internal:8788
export P2P_SHARED_DIR=/mnt/nfs/shared
./backend_app
```

Kubernetes Deployment

StatefulSet Configuration

```
apiVersion: apps/v1
kind: StatefulSet
metadata:
  name: loopline-backend
spec:
  serviceName: loopline-backend
  replicas: 3
  selector:
    matchLabels:
      app: loopline-backend
  template:
    metadata:
      labels:
        app: loopline-backend
    spec:
      containers:
        - name: backend
          image: loopline-backend:latest
          ports:
            - containerPort: 8787
              name: http
            - containerPort: 8788
              name: transfer
          env:
            - name: P2P_HTTP_PORT
              value: "8787"
            - name: P2P_TRANSFER_PORT
              value: "8788"
            - name: P2P_ADVERTISED_HOST
              valueFrom:
                fieldRef:
                  fieldPath: status.podIP
            - name: P2P_SYNC_PEERS
              value: "loopline-backend-0.loopline-backend:8788,loopline-backend-1.loopline-backend:8788,loopline-backend-2.loopline-backend:8788"
          volumeMounts:
```

```
      - name: shared
        mountPath: /shared
      - name: received
        mountPath: /backend/received
volumeClaimTemplates:
- metadata:
    name: shared
  spec:
    accessModes: [ "ReadWriteOnce" ]
    resources:
      requests:
        storage: 500Gi
- metadata:
    name: received
  spec:
    accessModes: [ "ReadWriteOnce" ]
    resources:
      requests:
        storage: 100Gi
```

Service Configuration

```
apiVersion: v1
kind: Service
metadata:
  name: loopline-backend
spec:
  clusterIP: None
  ports:
    - port: 8787
      name: http
    - port: 8788
      name: transfer
  selector:
    app: loopline-backend
---
apiVersion: v1
kind: Service
metadata:
  name: loopline-backend-lb
spec:
  type: LoadBalancer
  ports:
    - port: 80
      targetPort: 8787
      name: http
    - port: 8788
      targetPort: 8788
      name: transfer
  selector:
    app: loopline-backend
```

Monitoring & Logging

Metrics to Monitor

HTTP Server:

- Port 8787 availability
- Active connections count
- Request latency (p50, p95, p99)
- Error rate (4xx, 5xx)

Transfer Server:

- Port 8788 availability
- Active P2P connections
- Throughput (bytes/sec)
- Transfer success rate

Services:

- FileVaultService: Disk I/O, directory scans
- TransferService: Upload/download counts
- SharedSyncService: Sync lag, change detection

System:

- CPU usage
- Memory usage
- Disk usage (per directory)
- Network I/O

Health Check Endpoint (Recommended Addition)

```
GET /health
```

Response:

```
{
  "status": "healthy",
  "uptime_seconds": 3600,
  "active_connections": 12,
  "transfer_port_open": true,
  "sync_peers_connected": 2,
  "disk_usage": {
    "received": "45%",
    "sent": "12%",
    "shared": "78%"
  }
}
```

Troubleshooting

Common Issues

HTTP Server Won't Start

Error: Could not start HTTP server on 0.0.0.0:8787

Causes:

- Port already in use
- No permission to bind to port
- Network interface not available

Solution:

```
# Check port usage
lsof -i :8787 # macOS/Linux
netstat -ano | findstr :8787 # Windows

# Change port
./backend_app --http 9999
```

Peers Not Syncing

Symptoms: Shared folder not updating on remote nodes

Causes:

- Firewall blocking port 8788
- Incorrect peer address
- Network connectivity issue

Debug:

```
# Test connectivity
telnet peer-address 8788 # Check if reachable
ping peer-address       # Check network

# Check peer list
curl http://localhost:8787/api/peers # Get configured peers
```

File Transfers Failing

Symptoms: Files stuck in upload/download

Causes:

- Disk space full
- File permissions incorrect
- Transfer timeout

Solution:

```
# Check disk space
df -h /var/loopleftine/received

# Check permissions
chmod 755 /var/loopleftine/received
chmod 755 /var/loopleftine/sent
chmod 755 /var/loopleftine/shared

# Restart backend
kill $(pgrep backend_app)
./backend_app
```

Frontend Can't Connect to Backend

Error: Connection refused or CORS errors

Causes:

- Backend not running
- Wrong backend URL in frontend config
- CORS policy issues

Solution:

```
# Verify backend is running
curl http://localhost:8787/api/status

# Check frontend configuration
cat .env | grep REACT_APP_BACKEND_URL

# Add CORS headers to backend (if needed)
# Modify http_view.hpp to include:
# Access-Control-Allow-Origin: *
```

High Memory Usage

Symptoms: Backend process consuming excessive RAM

Causes:

- Large files in memory
- Memory leak in service
- Too many concurrent connections

Debug:

```
# Monitor process
top -p $(pgrep backend_app)
ps aux | grep backend_app

# Check open files
lsof -p $(pgrep backend_app)

# Reduce concurrent connections
# Add connection pool limit in http_server()
```

Disk Space Issues

Symptoms: Files not transferring, directories not created

Causes:

- Disk full
- Wrong mount point
- Permission denied

Solution:

```
# Check disk usage
du -sh /var/loopleftine/*

# Clean old files
find /var/loopleftine/received -mtime +30 -delete

# Change storage location
export P2P_RECEIVED_DIR=/mnt/larger_disk/received
```

Debug Mode

Enable verbose logging:

```
# Windows
set DEBUG=loopleftine:*
.\backend_app.exe

# Linux/macOS
export DEBUG=loopleftine:*
./backend_app
```

Performance Tuning

Recommended Settings by Use Case

Small Office (< 10 users, < 1TB data)

```
P2P_HTTP_PORT=8787
P2P_TRANSFER_PORT=8788
P2P_BIND_HOST=0.0.0.0
P2P_ALLOW_REMOTE=false # Restrict to local network
```

Medium Enterprise (10-100 users, 1-10TB data)

```
P2P_HTTP_PORT=8787
P2P_TRANSFER_PORT=8788
P2P_BIND_HOST=0.0.0.0
P2P_ALLOW_REMOTE=true
# Deploy on VM with 4GB RAM, 500GB SSD
```

Large Deployment (> 100 users, > 10TB data)

```
# Use Kubernetes cluster
# Replicas: 5+
# Storage: NFS mount for shared folder
# Load balancer: Nginx or cloud provider
```

Conclusion

This documentation provides a complete technical reference for the Loopline P2P file transfer and synchronization system. It covers:

- Backend architecture and components
- Frontend design and implementation
- API specifications
- Data flow and synchronization
- Configuration options
- Deployment strategies
- Troubleshooting guides
- Performance tuning

For updates, refer to the source code and commit history. Report issues and feature requests through the project's issue tracker.

Document Version: 1.0

Last Updated: May 25, 2026

Maintained By: Development Team